Project Title:

AI Football Analysis System (**Goal Sight**)

Team Members:

Name	ID	Program
Ahmed Sameh Ezzat	202202151	Software Development
Hesham Ashraf	202201477	Software Development
Mohamed Wael	202201445	Data Science and Artificial Intelligence (DSAI)
Ahmed Amr	202201404	Data Science and Artificial Intelligence (DSAI)

Supervisor:

Dr.Yousry Abdul Azeem

Semester / Year:

Fall / 2025

Team Number: #62

Evaluation Criteria:

1. Project Summary & Scope

The primary objective of the AI Football Analysis System is to democratize access to advanced sports analytics by providing a cost-effective, real-time solution for performance tracking. Currently, in-depth performance analytics are often manual, delayed, or restricted to elite clubs with high budgets, leaving a gap for smaller teams and individual players. This project directly addresses these stakeholder needs by utilizing computer vision and machine learning to analyze match footage, offering coaches and scouts the real-time data necessary for informed tactical decisions, while providing players with tools to track their physical and technical development. Additionally, the system enhances fan engagement by providing analytical insights that deepen their understanding of the game.

The project scope is strictly bounded to ensure feasibility within the academic timeline. The system encompasses two main platforms: a mobile application for professionals (coaches, scouts, and players) to view live match statistics and a website for fans to access post-match reports and historical data. Core "in-scope" features include real-time player tracking, event detection (goals, assists, and fouls), and the generation of interactive player profiles. To ensure realistic delivery, the scope explicitly excludes live audio/video commentary, predictive modeling for match outcomes or injuries, customizable user interfaces for fans, and support for sports other than soccer.

Progress Documentation Since the initial proposal, the team has established significant momentum, moving from theoretical research to concrete implementation and architectural design.

2. Progress Documentation

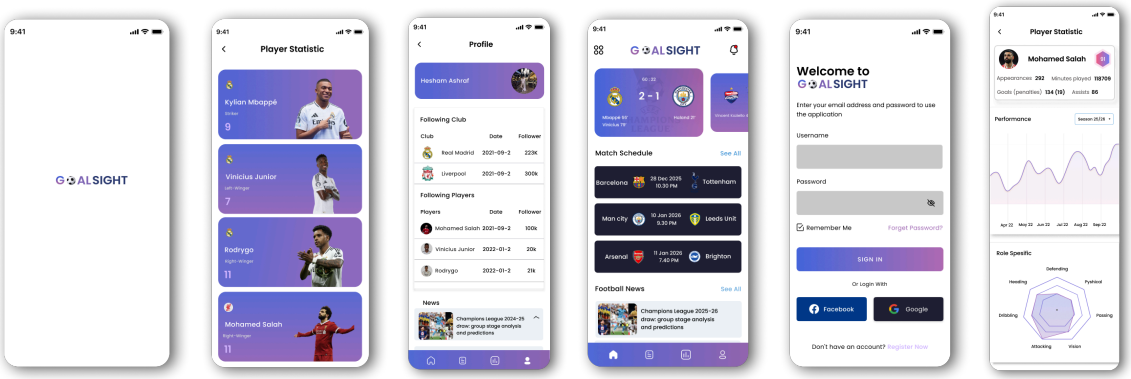
Key tasks completed since the proposal include:

Research & Requirement Analysis: Completed a detailed review of existing soccer analysis technologies and finalized the technical stack, selecting Python (OpenCV, TensorFlow) for AI processing and Flutter for the mobile application to ensure cross-platform high-performance rendering.

System Architecture Design: Defined a microservices architecture to ensure scalability, separating the system into independent modules for frontend, backend, and AI processing.

This structure has been visualized in detailed architectural diagrams, mapping the data flow between the Mobile App, Venue Dashboard, and Admin Dashboard.

Fig1: prototype



UI/UX Design & Prototyping: Designed high-fidelity wireframes and mockups for the mobile interface, establishing the visual identity and user flows for the "Coaches" and "Fans" dashboards.

Core Computer Vision Implementation (Code Committed):

Object Detection & Tracking: Implemented the Tracker class, integrating Ultralytics. YOLO for robust object detection and supervision.ByteTrack for persistent multi-object tracking. This module processes frames in batches to classify distinct entities (players, referees, ball) and manages state persistence using pickle stubs to optimize re-run performance.

Visual Data Annotation: Developed a custom annotation pipeline within the draw_annotations method to visualize tracking results directly on match footage. This features specialized OpenCV drawing logic (draw_ellipse, draw_triangle) to distinctively mark players with grounded ellipses and tracking IDs, while highlighting the ball's position with a dynamic triangular pointer.

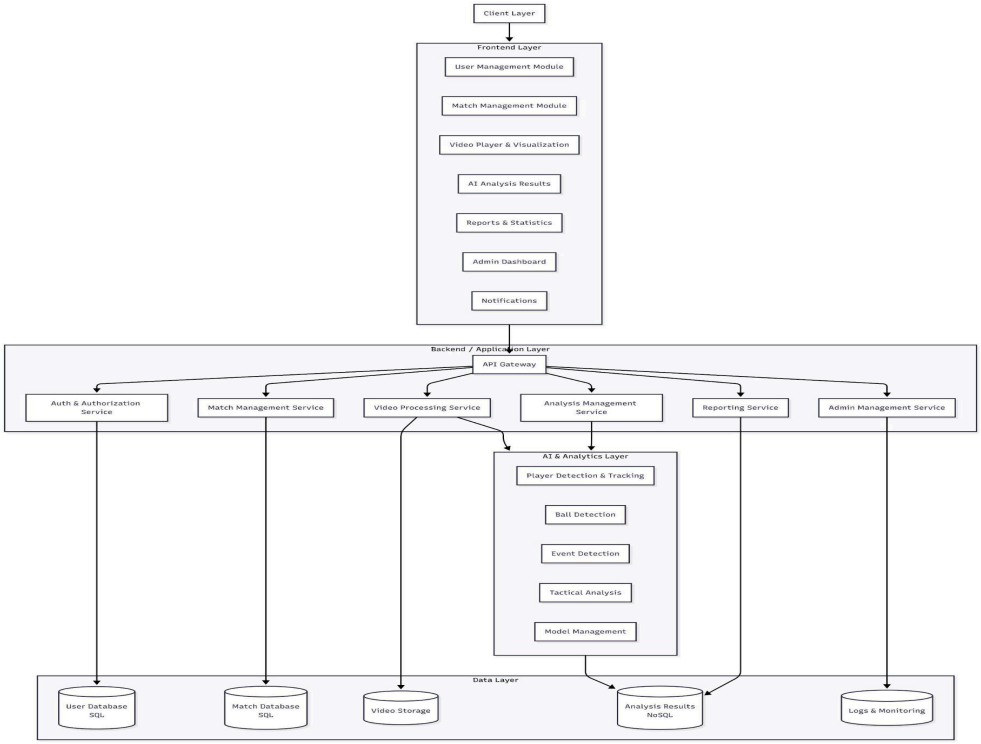
Ball Interpolation: Implemented the interpolate_ball_positions method within the tracking pipeline to address occlusion issues, ensuring a continuous and smooth ball trajectory by mathematically filling in gaps where detections are missing.

Team Assignment: Integrated the TeamAssigner module to automatically classify players into opposing teams. This functionality analyzes visual features (assign_team_color) from the initial frame and dynamically assigns a team identity to each player track throughout the match for tactical analysis.

Fig2 : detection:

2.2 System Architecture Diagram:

-fig 3: system architecture:



3. Component Breakdown

3.1. AI & Computer Vision Engine

- **Detail & Functionality:**
  - **Function**: It acts as the "eyes" of the system, ingesting match footage to automatically identify and follow players, referees, and the ball. It produces both raw analytical data and a visually annotated video stream for verification.
  - **Core Logic**: The engine performs a multi-stage pipeline:
    1. **Detection & Annotation**: It detects distinct entities in every frame, visualizing players with grounded ellipses and the ball with a dynamic green triangle pointer.
    2. **Tracking Persistence**: It assigns unique integer IDs (e.g., "12", "30") to each entity, ensuring that a specific player retains the same ID throughout the clip even as they move.
    3. **Team Differentiation**: It distinguishes between opposing teams and the referee by analyzing jersey colors and applying corresponding color-coded ellipses (e.g., black vs. yellow rings).

4. **Spatial Transformation**: Finally, it mathematically transforms the 2D screen coordinates into real-world pitch coordinates (meters) to calculate speed and distance.
    - **Inputs**: Raw video frames from the uploaded match footage.
    - **Outputs**: A structured stream of data (Player IDs, Team IDs, x,y coordinates) and an annotated output video featuring bounding boxes, tracking IDs, and team-colored ellipses.
- **Technology Stack:**
    - **Python**: Selected as the primary language due to its dominant ecosystem for AI and machine learning.
    - **OpenCV & TensorFlow/YOLO**: Used for the heavy lifting of image processing and deep learning object detection.
    - **NumPy**: Essential for the complex mathematical matrix operations required to map video pixels to field dimensions.
    -

3.2. Mobile Application (Frontend)

- **Detail & Functionality:**
    - **Function**: Provides a portable, interactive dashboard for viewing live match analytics and individual performance histories.
    - **Core Logic**: It authenticates users securely and maintains a real-time connection to the server. During a match, it renders data overlays (like player names and stats) directly on top of the video feed. It also requests and formats historical data into easy-to-read charts for player profiles.
    - **Inputs**: User interactions (taps, swipes) and real-time JSON data streams from the backend.
    - **Outputs**: Visualized statistics, video overlays, and interactive charts.
- **Technology Stack:**
    - **Flutter**: Chosen for its ability to compile to native code for both iOS and Android from a single codebase. It offers high-performance rendering (60fps), which is critical for smooth video overlays and real-time data visualization.
    - **WebSockets (Socket.IO)**: Required to establish the persistent, low-latency connection needed for live updates.

3.3. Web Dashboard (Frontend)

- **Detail & Functionality:**
    - **Function**: Serves two purposes: a fan engagement portal for detailed post-match reports and an administrative control panel for managing the system.
    - **Core Logic:** The Fan Dashboard visualizes complex datasets (heatmaps, passing networks) that require larger screens. The Admin Dashboard allows system managers to configure venues, manage subscription plans, and oversee user permissions.
    - **Inputs:** Administrator commands and massive datasets for historical matches.

- **Outputs:** Detailed analytical reports, system configuration updates, and user management logs.

- **Technology Stack:**
  - **React.js**: Facilitates a component-based architecture, making it easy to build complex, interactive dashboards that update dynamically without reloading the page.

3.4. Backend Server & API Gateway

- **Detail & Functionality:**
  - **Function**: It orchestrates the flow of data across the entire system, ensuring that the right data reaches the right user securely.
  - **Core Logic**: It hosts the API Gateway, which routes requests ("Login," "Get Match Data," etc.) to the appropriate modules. It manages distinct business logic modules like user management, venue management, and payment processing. It also handles the WebSocket "broadcasts" to push live data to mobile apps.
  - **Inputs**: HTTP REST requests from frontends and raw tracking data from the AI engine.
  - **Outputs**: JSON responses, authentication tokens, and real-time data broadcasts.
- **Technology Stack:**
  - **Node.js & Express**: Selected for their non-blocking, event-driven architecture, which is ideal for handling thousands of concurrent connections during a live match.

3.5. Data Persistence Layer (Database)

- **Detail & Functionality:**
  - **Function**: Securely stores two types of data: structured business data (users, teams, schedules) and high-volume analytical data (tracking coordinates).
  - **Core Logic**: The system separates concerns: critical business relationships (e.g., "User A belongs to Team B") are enforced with strict schemas, while the massive amounts of tracking data generated every second are stored in a format optimized for speed and flexibility.
  - **Inputs**: Account details, match schedules, and frame-by-frame tracking logs.
  - **Outputs**: Query results for login verification, historical stats, and match reports.
- **Technology Stack:**
  - **PostgreSQL**: Used for relational data (Users, Teams) to ensure data integrity.
  - **MongoDB**: (Hybrid approach) Used for storing the large, unstructured JSON logs generated by the AI tracking system.

3.6. Cloud Infrastructure

- **Detail & Functionality:**
  - **Function**: Provides the scalable computing power and storage capacity needed to process video and serve users globally.

- **Core Logic**: It manages the storage of large video files (Raw Footage) and hosts the virtual servers that run the AI models and Backend logic. It scales resources up or down based on demand.
- **Inputs**: Uploaded video files and server deployment scripts.
- **Outputs**: A reliable, 24/7 accessible hosting environment.

- **Technology Stack:**
  - **Google Cloud**: Industry-standard providers offering reliable Object Storage (Cloud Storage) for videos and Compute Instances (Compute Engine) for running the heavy AI workloads.

4. Data Flow

4.1 Data Acquisition & Ingestion

The flow begins when the user uploads raw match footage via the frontend. The Video Processing Service receives the file, validates the format, and stores the raw video in Cloud Storage. Simultaneously, match metadata (teams, date, venue) is normalized and stored in the SQL Database to serve as the reference key for the analysis session.

4.2 AI Processing & Transformation Pipeline

Upon upload completion, the core DSAI processing pipeline is triggered. The data undergoes a sequential transformation process:

1. **Frame Extraction & Object Detection:**
   The video is decomposed into individual frames. The Computer Vision Module (using YOLO) scans each frame to detect objects, generating initial data packets containing Bounding Boxes (x, y, w, h) and Class IDs (Player, Referee, Ball).
2. **Object Tracking (Data Association):**
   These raw detections are passed to the Tracker (ByteTrack), which associates detections across consecutive frames. This transforms isolated detection data into continuous Tracks, assigning a unique Track_ID to each entity to maintain identity persistence throughout the match.
3. **Feature Extraction & Team Assignment:**
   The Team Assigner extracts visual features (color histograms) from player bounding boxes in the initial frames. It clusters these features to determine team colors and tags each player track with a specific Team_ID, enriching the data with semantic context.
4. **Spatial Transformation & Stabilization:**
   To generate tactical metrics, the data must be converted from "Screen Space" (pixels) to "Pitch Space" (meters):
   - Camera Compensation: The Camera Movement Estimator calculates frame-to-frame optical flow to determine camera panning/zooming. It applies a stabilization vector to the tracking data to negate camera motion.
   - View Transformation: The View Transformer applies a perspective matrix (Homography) to the stabilized coordinates. Using cv2.perspectiveTransform, it maps the 2D pixel coordinates of players and the ball to 2D real-world coordinates on a tactical pitch model.

5. **Data Refinement (Interpolation):**
   The system identifies gaps in the ball tracking data caused by occlusion. The interpolation logic fits a curve to existing points to mathematically estimate and fill missing coordinates, ensuring a continuous trajectory.

4.3 Storage & Persistence

The fully processed data—now consisting of stabilized trajectories, team assignments, and real-world coordinates—is serialized (e.g., JSON/Pickle) and stored in the Analysis Results Database (NoSQL). This structure allows for efficient querying of high-volume spatiotemporal data.

4.4 Visualization & Consumption

When a user requests analysis:

1. The API Gateway fetches the processed coordinate data and matches metadata.
2. The frontend utilizes this structured data to render:
   - Overlays: Drawing ellipses and IDs directly onto the video frames using the coordinate data.
   - Tactical Views: Plotting the transformed real-world coordinates onto a 2D virtual pitch.
   - Statistics: Aggregating the movement data to calculate distance covered, speed, and heatmaps.

5. Work Breakdown Structure (WBS)

Task	Responsible Team Member	Start Date	End Date	Status
1. Research & System Design				
1.1 Requirement Gathering & Scope Definition	**All Members**	**01-Sep-25**	**15-Sep-25**	**Completed**
1.2 Technical Stack Selection (Python, Flutter, React, Node.js)	**All Members**	**16-Sep-25**	**20-Sep-25**	**Completed**

1.3 System Architecture Design (Microservices Diagrams)	Hesham Ashraf	21-Sep-25	10-Oct-25	Completed
2. AI & Computer Vision Implementation				
2.1 Object Detection Pipeline (YOLOv8 Integration)	Mohamed Wael	25-Oct-25	15-Nov-25	Completed
2.2 Tracking & ID Persistence (ByteTrack Implementation)	Mohamed Wael	10-Nov-25	20-Nov-25	Completed
2.3 View Transformer Logic (Pixel-to-Meter Mapping)	Ahmed Amr	15-Nov-25	30-Nov-25	In Progress
2.4 Coordinate Transformation (transform_point method)	Ahmed Amr	25-Nov-25	05-Dec-25	In Progress
2.5 Camera Movement Estimator (Optical Flow)	Mohamed Wael	01-Dec-25	10-Dec-25	In Progress
2.6 Team Assignment Module (Color Clustering)	Ahmed Amr	05-Dec-25	12-Dec-25	Completed

2.7 Ball Interpolation (Occlusion Handling)	Mohamed Wael	10-Dec-25	15-Dec-25	Completed
3. Backend & API Development				
3.1 Environment Setup (Node.js/Express + PostgreSQL)	Hesham Ashraf	15-Dec-25	05-Jan-26	In Progress
3.2 Authentication Module (JWT/OAuth2)	Ahmed Sameh	06-Jan-26	25-Jan-26	Pending
3.3 Video Upload API & Storage Config	Hesham Ashraf	26-Jan-26	10-Feb-26	Pending
3.4 WebSocket Integration (Real-time Streaming)	Ahmed Sameh	11-Feb-26	25-Feb-26	Pending
3.5 Data Aggregation Logic (Stats Calculation)	Hesham Ashraf	26-Feb-26	10-Mar-26	Pending
4. Frontend Development (Mobile & Web)				
4.1 Mobile UI Implementation (Flutter Screens)	Hesham Ashraf	15-Dec-25	30-Jan-26	In Progress

4.2 Video Overlay Rendering (CustomPainter)	Ahmed Sameh	01-Feb-26	20-Feb-26	Pending
4.3 Real-time Data Binding (WebSocket Client)	Ahmed Sameh	21-Feb-26	10-Mar-26	Pending
4.4 Web Dashboard (Admin & Reports Views)	Hesham Ashraf	11-Mar-26	30-Mar-26	Pending
5. Integration, Testing & Final Delivery				
5.1 System Integration (AI -> Backend -> Frontend)	All Members	01-Apr-26	15-Apr-26	Pending
5.2 Latency Testing & Performance Optimization	Mohamed Wael	16-Apr-26	25-Apr-26	Pending
5.3 Accuracy Validation & Ground Truth Comparison	Ahmed Sameh	16-Apr-26	25-Apr-26	Pending
5.4 Cloud Deployment (Google Cloud)	Hesham Ashraf	26-Apr-26	05-May-26	Pending
5.5 Final Thesis Writing & Defense Preparation	All Members	06-May-26	25-May-26	Pending

6. Teamwork & Equity

AI & Computer Vision Unit (DSAI Track):

- **Mohamed Wael:** Takes ownership of the Detection & Tracking Pipeline. His primary responsibility is optimizing the YOLO models and implementing the logic for object persistence (ByteTrack) , occlusion handling (interpolate_ball_positions) and object tracking.
- **Ahmed Amr:** Focuses on Spatial Logic & Transformation. He leads the development of the mathematical components, specifically the ViewTransformer and TeamAssigner modules, which convert raw pixel data into tactical pitch coordinates**.**

Software Development Unit (SWD Track):

- **Ahmed Sameh**: Acts as the System Architect & backend lead. He is responsible for the server infrastructure (Node.js), Authentication protocols (OAuth2), and the real-time WebSocket communication layer that bridges the AI engine with the frontend.
- **Hesham Ashraf:** Leads the Frontend Engineering & Data Persistence. His focus is on the User Interface (Flutter App & React Dashboard) and designing the database schemas (SQL/NoSQL) to efficiently store and retrieve the massive amounts of tracking data generated.

Collaborative Effort & Shared Responsibility While roles are specialized, the team shares responsibility for critical junction points. The **System Integration** phase requires close collaboration between the DSAI unit (defining data outputs) and the SWD unit (consuming data inputs). Furthermore, all members participate in **Requirement Gathering**, **Documentation**, and **Final Testing**, ensuring a unified vision for the product.

7. Risk Analysis & Mitigation

ID	**Risk Category**	Risk Description	Probability	Impact	Mitigation Strategy	Owner

R1	**Technical**	High Latency in Real-Time Processing: The AI pipeline (YOLO + Tracking + View Transformer) may take longer than 3 seconds to process frames, causing a significant delay in the "Live" feed.	Medium	High	1. Use lightweight YOLO models (e.g., YOLOv8n) for faster inference. 2. Implement batch processing optimization in my_tracker.py. 3. Run the AI engine on a GPU-enabled server (AWS g4dn instance) during the demo.	Mohamed Wael
R2	**Technical**	Occlusion & Tracking Loss: Players or the ball may be hidden behind other players, causing the tracker to lose the unique ID or the ball position to disappear.	High	High	1. Implemented: Use the interpolate_ball_positions logic to mathematically fill gaps in the ball's trajectory. 2. Use ByteTrack which is specifically designed to handle low-confidence detections during occlusion.	Mohamed Wael

R3	**Technical**	Camera Perspective Failure: The ViewTransformer relies on detecting specific pitch keypoints. Poor video quality or extreme camera angles might cause the cv2.getPerspectiveTransform function to fail.	Medium	High	1. Implement error handling to fallback to the last known valid transformation matrix. 2. Create a manual calibration tool in the Admin Dashboard to allow users to manually set the 4 pitch corners if automatic detection fails.	Ahmed Amr
R4	**Resource**	Flutter Overlay Complexity: The team may face a learning curve with Flutter's CustomPainter for drawing complex dynamic overlays (ellipses/triangles) over video at 60fps.	Medium	Medium	1. Prioritize learning standard Flutter graphics libraries early in Phase 4. 2. Fallback strategy: Draw the annotations on the backend using OpenCV (as currently implemented in draw_annotations) and stream the processed video instead of raw data, if client-side rendering proves too difficult.	Hesham Ashraf

R5	**Timeline**	Integration Delays: Connecting the Python AI engine with the Node.js backend via WebSockets may introduce unexpected bugs and data format mismatches, delaying the schedule.	Medium	High	1. Define a strict JSON schema for the data exchange early in Phase 3. 2. Schedule "System Integration" [Task 5.1] to start immediately after the API is ready, rather than waiting for the full frontend to be finished.	Ahmed Sameh
R6	**Resource**	GPU Cost & Availability: Training models and running the demo requires expensive GPU cloud instances, which might exceed the student budget.	Low	Medium	1. Develop and test primarily on local gaming laptops (Hybrid approach). 2. Utilize free tiers on Google Colab for model training.	All Members

8. Professional Communication

- **Documentation Standards & Visual Clarity:** The project exemplifies academic rigor through structured reporting and the use of industry-standard visualizations, utilizing high-fidelity Figma prototypes and architectural block diagrams to articulate complex system concepts with precision and clarity.
- **Operational Professionalism & Workflow:** A professional standard of execution is maintained through a robust collaborative framework that adheres to strict code quality guidelines (modular

Python architecture) and leverages version control (GitHub) alongside agile task tracking (Trello) to ensure consistent, error-free delivery.

GitHub Repository Link:

https://github.com/hesham-ashraf/Grad_Project_GOAL_SIGHT.git

Figma Prototype Link:

https://www.figma.com/design/n0UZo6np7vdAYxzPKikFtN/GoalSight?node-id=0-1&t=nQo9a20Kv0l6OFE8-1